

Workflows and Automation

Philip Trammell¹

February 10, 2026

Labor is typically bundled into jobs, comprising multiple tasks, instead of transacted by the task. This may be because performing a task increases one's productivity not only at the task itself but at related tasks. I show that these learning spillovers can introduce a convexity to the relationship between automation and output. Automating the first few tasks in a cluster with high spillovers (a “work-flow”) has little or no effect on output, since workers still perform some automatable tasks to increase their productivity at the non-automatable tasks. Automating the remainder of the tasks in such a cluster increases output more than automating the first few, as it becomes efficient to adopt the automation fully. Automation at a larger scale increases output by even more, if machines are intelligent enough also to “learn by doing” and can do a wider range of tasks than any single human.

¹Digital Economy Lab, Stanford University. Contact: trammell@stanford.edu. I thank Pamela Mishkin for the conversation that inspired this paper and Chris Tonetti, Peyman Shahidi, Basil Halperin, Chad Jones, and Enrique Ide for helpful feedback.

1 Introduction

The simple task framework. Call a *simple task model* one in which output exhibits constant returns to scale in the performance of a set of necessary tasks (see in particular Zeira (1998) and Aghion et al. (2019)).¹ In these models, each task can be performed by labor and/or capital. As long as capital is plentiful enough, labor performs only those tasks which capital cannot. Making more tasks automatable increases output by allowing labor to shift to the remaining non-automatable tasks, which are in shorter supply.

Simple task models have been widely used to forecast the impact on output of advances in AI. Aghion et al. (2019) use their task-based model to argue that AI may simply continue a long-running process in which constant growth is sustained by a constant process of task-by-task automation. Acemoglu (2025) combines a similar model with Eloundou et al.’s (2024) estimates of each O*NET task’s LLM exposure to forecast the GDP impact of LLMs. Aghion and Bunel (2024), responding to a draft of Acemoglu’s paper, contest practically all of Acemoglu’s assumptions except the central one: that output is a function of task quantities, where each task’s quantity is proportional to the capital or labor assigned to it.

The economic impacts of AI to date seem to have been much smaller than those predicted by a simple task model. This may be due to what is arguably the central limitation of such a model, namely that it does not explain the structure of work. In particular, it does not capture the gains from specialization (why it is efficient for each worker to perform a small fraction of all tasks), nor the limits to specialization (why almost every worker performs a set of rather disparate tasks, instead of a single task or set of almost identical tasks). This paper presents a minimally modified task model that can accommodate these two facts about how work is structured, and considers how we should revise our analysis of the impact of advances in task-level automation in light of it.²

¹Unlike e.g. Acemoglu and Restrepo (2018), we will suppose the set of tasks is fixed.

²Autor and Thompson (2025) and Freund and Mann (2025) study the labor market impacts of task automation, especially from AI, under the assumption that each job consists of exogenously fixed bundle of tasks, or the subset of these that is not yet automated. I offer a stylized account of why jobs are composed as they are, given the tasks that are and are not automatable. The model predicts various phenomena which cannot occur in a simple task model with exogenous bundling, e.g. that it may be

Specialization due to learning by doing. The standard account of specialization, dating at least to [Smith \(1776\)](#) and formalized by [Yang and Borland \(1991\)](#), [Becker and Murphy \(1992\)](#), and much subsequent work is that when a worker performs a task many times, he is more productive at it. This productivity may come directly from learning by doing or from the fact that more doing motivates more education. As [Becker and Murphy \(1992\)](#) show, both can be modeled for our purposes as learning by doing, as I will do for simplicity.

Literal learning by doing, in turn, may consist of learning general lessons about how to perform a task efficiently, or of the knowledge of the end of a project that comes from having performed the beginning. Writing may be a single task, but an author will write the second chapter of a book much better if she wrote the first chapter, even if writing the first chapter hardly improved her as a writer in general.³

Task clustering due to learning spillovers. Despite the benefits of specialization, we are far from working in a fully “Taylorized” economy in which everyone continually performs a single task with minor modifications. [Lindbeck and Snower \(2000\)](#) propose that this is due to *learning spillovers*: each worker is more productive at a task when she also performs related tasks, so excessive specialization can decrease productivity.

Consider the O*NET occupation of Economist. One of its 14 associated tasks (#7538) is to “analyze... data to explain economic phenomena”. Another (#21106) is to “[e]xplain [the] economic impact of policies to the public”. Though data analysis and public communication are in some sense very distinct tasks, they cannot easily be assigned to different people: analyzing the data makes an economist much better

efficient for labor to perform some automatable tasks for their learning spillovers.

³The latter kind of learning by doing can straightforwardly account for the fact that in many occupations, compensation increases superlinearly in hours worked. [Ameriks et al. \(2020\)](#) find that older Americans would generally prefer to reduce their working hours gradually if it were possible to do so without reducing their hourly wage, instead of moving discretely from full-time work to retirement, implying that the relative dearth of high-paying part-time work is a feature of the production function and not of worker preferences. Recent work by [Jarosch et al. \(2025\)](#) analogously finds that workers in Germany and the UK (though interestingly not the US) would prefer fewer hours without sacrificing hourly pay. Intuitively, the very long hours often worked in law, consulting, and medical residencies seem attributable in part to the difficulty of partitioning large, time-sensitive projects across multiple employees rather than entirely to employee preferences for long hours: see e.g. [Prescott et al.’s \(2009\)](#) model in which workers choose workweek length given a potentially superlinear function from hours worked to labor services provided.

at credibly discussing its implications. Or consider the Computer Systems Engineer: “[v]erify[ing the] stability, interoperability, portability, security, or scalability of [a] system architecture” (#14669) is easier when one is also the architect (#14676, #14689).⁴

In short, performing task A may increase our productivity at task B. On some margins, it may do so by even more than performing more of task B. It may thus be efficient for each worker to perform multiple tasks.

The two examples above illustrate learning spillovers within the context of a particular work project, but they may also appear across projects, in a sense more reminiscent of conventional learning by doing. For example, one’s productivity in teaching may increase in both the number of hours one has spent teaching and the number of hours one has spent doing research, including research on subjects other than the one being taught.

Task automation given learning. Given learning, within and across tasks, automating a small number of tasks in isolation may have macroeconomic impacts greater or less than in a simple task model, depending on the structure of learning spillovers. In particular, technology that can automate just a few tasks in a task-cluster with high learning spillovers may have little or no effect on output, because adopting the technology would deprive workers of valuable learning. Suppose a coder splits her day evenly between coding and debugging, and an AI tool can only automate the coding. If the coder is less than half as productive at debugging code she did not write, it is not worth adopting the tool at all.

In the limit of full automation, however, the impact of automation is unambiguously greater with learning by doing than without, given intelligent machines that can do more (or learn from more instances of “doing”) than any single human worker. Because machines in this case are performing and learning from every task, this holds regardless of the structure of learning spillovers.

Thus, when we observe task-level automation having little impact on output today, this may not indicate that the predictions of a simple task model will always be too aggressive. Instead, it may indicate that, due to learning, output is highly convex in

⁴Shen and Tamkin (2026) find that using AI for coding “impairs conceptual understanding, code reading, and debugging abilities” and thus fails to deliver “significant efficiency gains on average”, despite its obvious speed at coding tasks in isolation.

the fraction of tasks automated.

Maximizing learning spillovers vs. minimizing handoff costs. Another explanation for task clustering is that splitting a project across many workers incurs coordination costs, especially communication costs. As a result, it may be efficient to assign a worker some task, even if in isolation it could be done more cheaply by another worker, because there are costs to temporarily transferring the project in which the task is embedded. A large literature studies the implications of these costs for optimal firm and team structure.⁵ [Demirer et al. \(2025\)](#) study the impact of task automation given these costs, which they call handoff costs. This framing lets them study the impact of task automation on microeconomic questions including skill premia and firm structure, applying insights from the literature cited above.

The “learning spillover” and “handoff cost” models of task clustering are very similar in spirit. Indeed, foregone handoff costs can be interpreted as a kind of learning spillover. We might say that a coder is more productive at debugging code she has written than code she has not, for example, because she does not have to pay the handoff cost of reading her own documentation.

The learning spillover framing, however, is arguably more useful when considering the macroeconomic impact of task automation. This is because learning offers an immediate explanation for the “first fact” about the structure of work, namely that there is specialization to begin with. A learning model thus allows task automation not only to attenuate but also magnify the useful learning that workers gain on the job. When applied to the case of intelligent machines, furthermore, a learning model sheds light on how automation may ultimately increase economic returns to scale, by allowing machines to learn from ever wider corpora of experience, as humans cannot ([Farboodi et al., 2025](#)). By contrast, in a model in which the structure of work is explained entirely by coordination costs, because the costs can only fall to zero, the returns to scale are constant in the limit when all work is done by machines for which these costs are eliminated.⁶

⁵See e.g. [Yang and Borland \(1991\)](#), [Becker and Murphy \(1992\)](#), [Bolton and Dewatripont \(1994\)](#), [Garricano \(2000\)](#), and [Dessein and Santos \(2006\)](#).

⁶A model can separately feature both specialization due to task-level learning and limits on specialization due to coordination costs ([Becker and Murphy, 1992](#)). Such a model would produce conclusions similar to those presented here, but arguably be less parsimonious.

Outline. I introduce a simple model of learning spillovers. The space of tasks is partitioned into “workflows”. Within a workflow, learning spillovers are strong, specialization is counterproductive, the economic impact of partial automation is smaller than in a simple task model. Across workflows, learning spillovers are weak, specialization is valuable, and the economic impact of automation is greater than in a simple task model.

In the simplest setting, introduced in Section 2, there are no cross-workflow spillovers. Furthermore, the learning drawn from tasks within a workflow is Leontief: i.e. it exhibits the strongest possible diminishing returns. It is thus efficient for each worker to perform (one or more) complete workflows rather than an arbitrary assortment of tasks.

I then consider the implications of this model for the GDP impact of advances in task automation. I find that its predictions differ from those of a simple task model in two ways.

1. Making the first few tasks in a workflow automatable increases output less than in a simple task model. This is because it remains efficient for workers to perform some automatable tasks—i.e. to forego some automation—to increase their productivity at the non-automatable tasks.
2. Making later tasks in a workflow automatable increases output more than earlier tasks, as it becomes efficient to automate all automatable tasks, as in a simple task model.

In short, I argue that the relationship between automation and growth, within each cluster with strong learning spillovers, should exhibit a convexity not found in simple task models.

In Section 3, I extend the model by weakening the distinction between tasks that are and are not in the same workflow. In particular, I allow the performance of tasks in workflow A to improve a worker’s productivity in workflow B to an intermediate degree, which decreases continuously with the “distance” between A and B. This extension accommodates the intuition that, even though a worker may perform multiple workflows—so that there is room for further gains from specialization—it is efficient for each job to consist of a cluster of adjacent workflows instead of an arbitrary assortment. This extended model preserves both the results above for the GDP impact

of automation. Furthermore, arguably a key distinction between AI-based and earlier waves of automation is that intelligent machines can themselves learn as they work, either continuously or because widespread deployment of one generation of AI and robotics systems facilitates the data collection used to train the next generation. By accounting for this, the extended model also yields a third result:

3. Automation speeds growth by even more than in a simple model in the limit, given that the machines can do more (or learn from more instances of “doing”) than any single human worker.

Machines can increase their productivity at a workflow not only as a result of performing its tasks at a large scale but also as a result of performing tasks in other workflows.

Section 5 concludes with a discussion of how models without learning may be leading us astray about the impact of automation in practice, and how a deeper understanding of how tasks and workflows compose into useful work might be valuable.

2 Basic model

Model. The global set of tasks is partitioned into workflows. Each workflow consists of a unit continuum of tasks. We will begin with the case in which there is only one workflow, as this is most similar to the setup of a simple task model.

Tasks i range from 0 to 1. For some threshold $T < 1$, tasks $i \leq T$ are *automatable*, in that they can be performed by capital or labor. Tasks $i > T$ can only be performed by labor. K_i and ℓ_i are the (integrable) densities of capital and labor assigned to task i respectively. For simplicity, we will say that output Y (equivalently, the output of the workflow) is Leontief in the quantities of the tasks Y_i , and that there is only worker.

The above is simply the Leontief case of a simple task model. To introduce learning spillovers within the workflow, we will say that an individual’s productivity at task i , A_i , increases in her performance of tasks $j \leq i$. In particular, we will say that

$$A_i = \left(\inf_{j \leq i} \ell_j \right)^\alpha, \quad \alpha > 0. \tag{1}$$

Note that the $\alpha = 0$ case corresponds to the simple case without learning.

Given capital stock K , an allocation of capital and labor across tasks is efficient if

it maximizes

$$Y \equiv \inf_i Y_i \quad (2)$$

subject to

$$Y_i = \begin{cases} K_i + A_i \ell_i, & i \leq T; \\ A_i \ell_i, & i > T; \end{cases} \quad (3)$$

$$\int_0^1 K_i di \leq K, \quad \int_0^1 \ell_i di \leq \ell, \quad (4)$$

where ℓ is the (exogenous) quantity of labor the worker supplies.

Finally, to simplify even further, we will assume that as long as $T < 1$, lack of capital is not a constraint on the margin. Nevertheless, given learning, it may be efficient for our worker to spend fraction $F > 0$ of her total labor on automatable tasks. The direct value of performing these tasks is zero, as she simply displaces the capital that could otherwise have performed them; the value is entirely the productivity gained on the remaining tasks.

Proposition 1 (The optimal allocation).

Given tasks up to T automatable, it is uniquely optimal⁷ to set

$$\ell_i = \begin{cases} \frac{F(T)}{T} \ell, & i \leq T; \\ \frac{1-F(T)}{1-T} \ell, & i > T, \end{cases} \quad (5)$$

yielding

$$Y = \left(\frac{F(T)}{T} \right)^\alpha \frac{1-F(T)}{1-T} \ell^{1+\alpha}, \quad (6)$$

where

$$F(T) = \min \left(T, \frac{\alpha}{1+\alpha} \right). \quad (7)$$

Proof. Given that the worker spends fraction $F \in [0, 1]$ of her time on automatable

⁷Up to null sets.

tasks, it is uniquely optimal (up to null sets) to set

$$\ell_i = \begin{cases} \frac{F}{T}\ell, & i \leq T; \\ \frac{1-F}{1-T}\ell, & i > T. \end{cases} \quad (8)$$

An unequal allocation of $F\ell$ to automatable tasks features $\ell_i < F\ell/T$ for some $i \leq T$, and thus lower A_i (1) for $i > T$ than under allocation (8). An alternative allocation of $(1-F)\ell$ to non-automatable tasks must feature a positive-measure task set $S \subset (T, 1]$ such that $\ell_i < \frac{1-F}{1-T}\ell$ for all $i \in S$ and some $i \leq \inf S$: so for all $i \in S$, A_i is weakly lower and ℓ_i strictly lower than in allocation (8).

Likewise, $F > T$ would require a positive-measure task set $S \subset (T, 1]$ such that $\ell_i < \frac{1-F}{1-T}\ell$ ($< \frac{F}{T}\ell$) for all $i \in S$ and some $i \leq \inf S$, leaving both A_i and ℓ_i strictly lower than in allocation (8) with $F \leq T$.

Since allocation (8) with $F \leq T$ yields output

$$\left(\frac{F}{T}\right)^\alpha \frac{1-F}{1-T} \ell^{1+\alpha},$$

it only remains to find the $F \in [0, T]$ that maximizes the above given T . □

How learning changes automation's impact on output. In a simple task model, the elasticity of output to increases in automatability—more precisely,

$$-\frac{d \ln(Y)}{d \ln(1-T)}$$

—equals 1 (in the long run, or immediately given unlimited capital).

Here, from equation (7), we see that when $T < \frac{\alpha}{1+\alpha}$, it is not efficient to use capital at all, because performing the automatable tasks gives the worker much more context on the remainder of the workflow and performing them costs the worker only a small fraction of her time. The elasticity of output to increases in automatability is 0. When $T > \frac{\alpha}{1+\alpha}$, on the other hand, substituting (7) into (6) yields

$$Y = \frac{\alpha^\alpha}{(1+\alpha)^{1+\alpha}} \frac{T^{-\alpha}}{1-T} \ell^{1+\alpha}, \quad (9)$$

whose local elasticity to $1 - T$ equals $1 - \alpha \frac{1-T}{T}$, which is between 0 and 1 and rises to 1 as $T \rightarrow 1$.

To compare the total impact of automation on output here with that in a simple task model, recall that in the latter, automation up to $T < 1$ multiplies output by $1/(1 - T)$. The proportion of this effect retained here—

$$\frac{Y_{\text{learning}}(T)/Y_{\text{learning}}(0)}{Y_{\text{simple}}(T)/Y_{\text{simple}}(0)}$$

—equals $1 - T$ for $T \leq \alpha/(1 + \alpha)$. It declines further in T for $T > \alpha/(1 + \alpha)$, as can be found by dividing (9) by $\ell^{1+\alpha}/(1 - T)$ to get

$$\frac{\alpha^\alpha}{(1 + \alpha)^{1+\alpha}} T^{-\alpha},$$

to an infimum of $\alpha^\alpha/(1 + \alpha)^{1+\alpha}$ as $T \rightarrow 1$. Doing provides learning, and more automation takes more learning away.

Specialization and multiple workflows. The model of this section offers a simple explanation for why it may be efficient for each worker to perform a suite of tasks.

It also offers an explanation for why wages might increase in hours worked. If $T = 0$, for example, a worker working ℓ hours per week (or with ℓ hours of cumulative experience, on the model's “experience”-based interpretation) spreads these hours equally across all tasks within the workflow, enjoying productivity ℓ^α at each task and thus producing $\ell^{1+\alpha}$ units of output.

With only one workflow, the model predicts that each worker will perform *all* tasks, as long as T is low. This is of course unrealistic, and fails to explain the gains from specialization. Suppose instead therefore that there are W symmetric workflows and N individuals, with $W \gg N$. Let Y_w denote the output of workflow w , and again for simplicity, let output $Y(Y_1, \dots, Y_W)$ be symmetric and Leontief in each Y_w . Then, fixing each individual's labor supply at 1, it is uniquely efficient for each individual to perform W/N workflows, dedicating N/W units of labor to each.

Absent automation, the output of each task, and aggregate output, then equals $(N/W)^{1+\alpha}$ so aggregate output per person equals $N^\alpha/W^{1+\alpha}$. This increases in N because a larger population allows each worker to specialize in fewer workflows.

If, for all w , tasks w_i with $i \leq T$ are automatable, the optimal allocation of individuals to workflows does not change. The optimal allocation of each individual's work within her workflows is still characterized by (5) and (7), and output still equals (6), with $\ell = N/W$.

3 A richer task space

Motivation. The model of the previous section offers a simple explanation of specialization and its limits, but it predicts that it is efficient for each worker to perform an arbitrary set of workflows, rather than a related set. It also predicts, implausibly, that the economic returns to scale from proportionally increasing the population will be as large as those from proportionally increasing each worker's work experience (α). These results are artifacts of the simplifying assumption that tasks are either entirely necessary for learning (if they are downstream in the same workflow) or entirely irrelevant (if they are in different workflows). This section weakens that assumption. Qualitatively, the central points are that:

- Insofar as we learn more by doing a given task (or related tasks) more than by doing more distant tasks, it is efficient to specialize and reap the resulting increasing returns.
- Insofar as there are diminishing returns to learning from a given task in isolation, specialization can be counterproductive. This is true regardless of the state of automation, but it means that feasible automation can be inefficient by requiring us to specialize in inefficient ways.

These points can be separated simply in a two-dimensional task space.

Allowing the model to feature specialization despite some degree of learning spillovers across the economy is also valuable for illustrating a potential implication of automation. As shown in the next section, an automated economy will exhibit more strongly increasing returns to scale than observed presently, insofar as machines can learn from work done by machines throughout the economy and humans only learn from their own narrow experience.

Model. There is an interval of workflows $w \in (0, 1]$, each of which contains a unit

interval of tasks wi . As in Section 2, output is Leontief in tasks (or equivalently, in the output Y_w of each workflow, which is Leontief in the output of each of its tasks). There are N workers, indexed by n . ℓ_{wi}^n denotes the density of n 's performance of task wi and L_w^n denotes the density of n 's performance of tasks in workflow w .⁸ Worker n exogenously supplies one unit of labor:

$$\int_0^1 \int_0^1 \ell_{wi}^n di dw = \int_0^1 L_w^n dw = 1. \quad (10)$$

Now, however, let the productivity of worker n at task wi equal

$$A_{wi}^n = (A_w^n)^{1+\alpha} \left(\inf_{j \leq i} \ell_{wj}^n \right)^\alpha, \quad \alpha > 0; \quad (11)$$

$$A_w^n = \left(\int_0^w (w-v)^{-\beta} L_v^n dv \right)^\gamma, \quad \gamma \in (0, 1), \quad \beta \equiv \frac{\gamma}{1+2\gamma}. \quad (12)$$

Identity (11) is constructed so that $A_{wi}^n \ell_{wi}^n$ is equalized, across tasks in different workflows performed by n , when $A_w^n L_w^n$ is so equalized (and n 's labor is allocated uniformly within each workflow). As a result, if each workflow w is performed by a single worker $n(w)$, output of w reduces to

$$Y_w = (A_w^{n(w)} L_w^{n(w)})^{1+\alpha}.$$

Because $(\cdot)^{-\beta}$ is decreasing, (12) ensures that more distant workflows $v \ll w$ contribute less to productivity at w .⁹

In this setup, a worker's productivity at workflow w increases only in her performance of workflows $v < w$. Our model of cross-workflow learning is closely analogous in this way to our model of learning spillovers within a workflow. This is mainly motivated by tractability. If cross-workflow learning occurs in both directions, with contributions to learning decreasing in $|w-v|$, the obvious qualitative result is maintained—it is efficient for each person to perform an interval of workflows (see below)—but it is

⁸Each worker's allocation of labor across workflows $L_{(\cdot)}^n$ is left-continuous and has finite $\|L_{(\cdot)}^n\|_2$, i.e. $\int_0^1 (L_w^n)^2 dw < \infty$. Since $\beta < 1/2$, this ensures that A_w^n is finite for all w and that the problem is well-behaved.

⁹We could alternatively assume that sufficiently distant workflows offer each other no learning spillovers at all. Given a population of N , no one will perform a pair of tasks more than $1/N$ apart in equilibrium, so this would make no difference.

more difficult to produce a model in which an optimal allocation can be found analytically.

Specialization into intervals of workflows. Here, the learning contributions of workflows $v_1, v_2 < w$ to productivity at w combine additively, and so are perfect substitutes: the opposite extreme of the Leontief learning maintained within a workflow. As a result, since $(\cdot)^{-\beta}$ is decreasing, efficiency requires partitioning the set of workflows into N equal intervals, each performed exclusively by a single worker.

Proposition 2 (The optimal allocation across workflows).

Absent automation, there is a unique optimal allocation,¹⁰ in which

- *the set of workflows is partitioned into N equal intervals, each performed exclusively by a single worker;*
- *given that worker n performs workflows $(\underline{w}, \bar{w}]$,*

$$L_w^n = (1 - \beta) (\bar{w} - \underline{w})^{\beta-1} (w - \underline{w})^{-\beta},$$

$$\text{so } A_w^n = A^* (\bar{w} - \underline{w})^{\gamma(\beta-1)} (w - \underline{w})^\beta \quad \text{for } A^* \equiv \left((1 - \beta) \frac{(\Gamma(1 - \beta))^2}{\Gamma(2 - 2\beta)} \right)^\gamma,$$

$$\text{so } A_w^n L_w^n = A^* (1 - \beta) (\bar{w} - \underline{w})^{(1+\gamma)(\beta-1)} \quad \forall w \in (\underline{w}, \bar{w}];$$

and thus

$$\bullet Y = (A^* (1 - \beta))^{1+\alpha} N^{(1+\gamma)(1-\beta)(1+\alpha)}.$$

Proof. It is hopefully intuitive that, in this setting,

- i. there is an optimal allocation;
 - ii. in any optimal allocation, each workflow is performed exclusively by a single worker; and
 - iii. in any optimal allocation, Y_w , and thus $A_w^n L_w^n$, must be equal across workflows.
- For a proof that the above allocation alone satisfies (ii) and (iii), and so is uniquely optimal by (i), see Appendix A.1. For a proof that (i) an optimal allocation does in fact exist and satisfy (ii) and (iii), see the [supplement](#). □

¹⁰Up to relabeling of the workers.

Within a workflow, absent automation, labor is of course still allocated equally across tasks.

The three components of the exponent on N reflect, respectively, the benefits of cross-workflow learning, captured by γ ; the costs of restricting learning to a narrower range of workflows, captured by $-\beta$; and the benefits of within-workflow learning, captured by α .

Automation. Suppose that, across all workflows, tasks wi with $i \leq T$ are automatable. Then the results of this section and the previous section are unchanged. It is still optimal to assign each worker an interval of workflows and to allocate $L^n (= 1)$ so that $A_w^n L_w^n$ is equalized across the interval. For purposes of allocating labor within workflow w , the $(A_w^n)^{1+\alpha}$ coefficient in (11) can be ignored: the allocation that equalizes $A_{wi}^n \ell_{wi}^n$ across i , and the impact on output of increasing T , are precisely as described in Section 2.

If instead tasks wi with $w \leq T_W$ are automatable, for some $T_W \in (0, 1)$, these are fully automated, since it is always more efficient to allocate a population across a narrower range of workflows. A proportional decrease to $1 - T_W$ functions like a proportional increase to N .

4 Machine learning by doing and the returns to scale

We will now briefly explore the third and last implication of learning for automation: that once automation is sufficiently advanced, if machines can learn by doing as humans can, growth will rise by *more* than in a simple task model.

We have been assuming for simplicity that output is Leontief across tasks and that, while $T < 1$, output is bottlenecked by labor and not by the tasks capital can perform. In our setting, before full automation, capital's productivity at tasks wi with $i < T$ is irrelevant. In particular, it is irrelevant whether we model capital in the traditional, learning-free way or as learning from the tasks it performs. "Machine learning by doing"¹¹ thus has interesting implications here only after full automation, so we will

¹¹I.e. some process by which the data gathered while doing work—including human evaluations—improves subsequent performance. This may include both continuous learning and simply the use of data gathered on the job in training future AI models.

consider its implications only in this case. It will hopefully be clear that similar implications can emerge more continuously in a model in which capital partially substitutes for labor on the margin while work remains only partially automated.

In a simple task model, output after full automation is AK . Once the labor bottleneck is fully relieved, output grows exponentially in the absence of further technological development (given positive net saving), and new learning can speed growth only by increasing the replication rate of capital. Here, suppose that capital's productivity at workflow w (A_w) and at task wi (A_{wi}) are given by (11)–(12), with K and k_{wj} in place of L^n and ℓ_{wj}^n . Instead of the unit labor supply constraint faced by each individual in (10), capital faces the constraint

$$\int_0^1 \int_0^1 k_{wi} di dw = \int_0^1 K_w dw = K.$$

The entire capital stock thus learns collectively, without being affected by fragmentation across workflows.

Then output exhibits increasing returns in capital. In particular, recalling that in this setting $(\underline{w}, \overline{w}] = (0, 1]$, we have

$$\begin{aligned} K_w &= K(1 - \beta)w^{-\beta} \\ \implies A_w &\propto K^\gamma w^\beta \implies Y \propto K^{(1+\gamma)(1+\alpha)}. \end{aligned}$$

The elasticity of output to capital is $(1 + \gamma)(1 + \alpha)$, even greater than the $(1 + \gamma)(1 - \beta)(1 + \alpha)$ elasticity of output to population that follows from Proposition 2.

This higher elasticity arises because, unlike in the human case, every unit of capital can learn from the experience of every other unit. When capital performs all workflows, the optimal allocation of a doubled capital stock doubles the density of capital on each workflow. By contrast, a doubled population must divide itself across workflows more finely, so that the mind performing each task benefits from a narrower range of learning.

5 Extrapolating task productivity gains from AI

Eloundou et al. (2024) evaluate the extent to which GPT-4, perhaps accompanied by

necessary “wrapper” software, can automate each task in the O*NET database. At face value, the analysis implies that a lightly augmented GPT-4 could automate 38% of the wage bill.¹² Since GPT-4 does not seem on track to increase output by anything close to 38%, recent attempts to quantify the GDP impact of LLMs (Acemoglu, 2025; Aghion and Bunel, 2024) have produced more intuitively reasonable figures largely by assuming, somewhat ad hoc,

- that tasks judged less than “75% automatable” (Eloundou et al. categories 0–2) generate no labor savings at all and
- that for tasks judged at least “75% automatable” (Eloundou et al. categories 3 and 4), their labor savings in practice will permanently equal their judged automatability multiplied by recent empirical estimates of time savings on a handful of tasks in real-world settings.

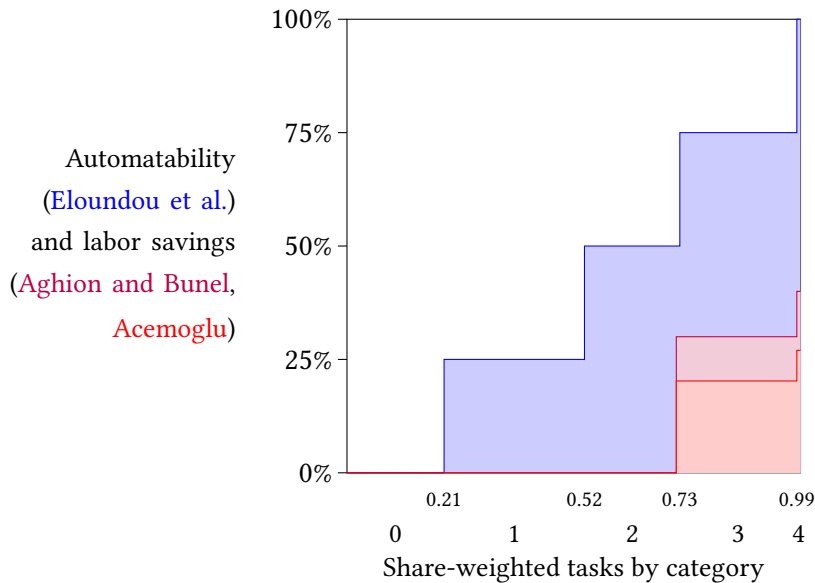


Figure 1: Distribution of “automatability” (Eloundou et al.) and “labor cost savings” (Acemoglu, Aghion and Bunel) across share-weighted tasks

After these and other adjustments, the impact on GDP, even of systems that can truly automate 38% of wage-adjusted tasks in isolation, is estimated to be a few percent at

¹²Following Acemoglu (2025), I am weighting each task by its occupation’s share of the 2019–2022 wage bill.

most.

These adjustments perhaps produce reasonable estimates of the impacts of LLMs on GDP today and in the very near future. However, because they permanently multiply task-level estimates of automatability by a factor of 0.27–0.4 even for *fully automatable tasks*, they have the strange implication that full automation would only yield 27–40% labor savings.

By contrast suppose that, as in the model of this paper, the lack of full labor savings on fully automatable tasks is due to the fact that some labor on task A remains necessary for the worker to understand how to integrate its output with the not-yet-automated task B (and indeed that for tasks with sufficiently low automatability, perhaps it is not worth automating at all, as the authors above assume). Then labor savings per automatable task will rise as more tasks become automatable. As AI advances, the GDP impact of AI will rise, perhaps quickly, to that suggested by a plain reading of [Eloundou et al.](#) or beyond.

References

- Acemoglu, Daron**, “The Simple Macroeconomics of AI,” *Economic Policy*, 2025, 40 (121), 13–58.
- **and Pascual Restrepo**, “The Race Between Man and Machine: Implications of Technology for Growth, Factor Shares, and Employment,” *American Economic Review*, 2018, 108 (6), 1488–1542.
- Aghion, Philippe and Simon Bunel**, “AI and Growth: Where Do We Stand?,” 2024. Working paper.
- , **Benjamin F. Jones, and Charles I. Jones**, “Artificial Intelligence and Economic Growth,” in Ajay Agrawal, Joshua Gans, and Avi Goldfarb, eds., *The Economics of Artificial Intelligence: An Agenda*, National Bureau of Economic Research, 2019, pp. 237–282.
- Ameriks, John, Joseph Briggs, Andrew Caplin, Minjoon Lee, Matthew D. Shapiro, and Christopher Tonetti**, “Older Americans Would Work Longer if Jobs Were Flexible,” *American Economic Journal: Macroeconomics*, 2020, 12 (1), 174–209.
- Autor, David and Neil Thompson**, “Expertise,” 2025.
- Becker, Gary S. and Kevin M. Murphy**, “The Division of Labor, Coordination Costs, and Knowledge,” *Quarterly Journal of Economics*, 1992, 107 (4), 1137–60.

- Bolton, Patrick and Mathias Dewatripont**, “The Firm as a Communication Network,” *The Quarterly Journal of Economics*, 1994, 109 (4), 809–39.
- Demirer, Mert, John J. Horton, Nicole Immorlica, Brendan Lucier, and Peyman Shahidi**, “The Economic Impacts of Generative AI on the Structure of Work,” 2025. Working paper.
- Dessein, Wouter and Tano Santos**, “Adaptive Organizations,” *Journal of Political Economy*, 2006, 114 (5), 956–95.
- Eloundou, Tyna, Sam Manning, Pamela Mishkin, and Daniel Rock**, “GPTs are GPTs: Labor market impact potential of LLMs,” *Science*, 2024, 384 (6702), 1306–8.
- Farboodi, Maryam, Andrew Koh, and Anchi Xia**, “Data-driven Automation,” 2025.
- Freund, Lukas and Lukas Mann**, “Job Transformation, Specialization, and the Labor Market Effects of AI,” 2025.
- Garicano, Luis**, “Hierarchies and the Organization of Knowledge in Production,” *Journal of Political Economy*, 2000, 108 (5), 874–904.
- Jarosch, Gregor, Laura Pilossoph, and Anthony Swaminathan**, “Should Friday be the New Saturday? Hours Worked and Hours Wanted,” 2025. NBER working paper 33577.
- Lindbeck, Assar and Dennis J. Snower**, “Multitask Learning and the Reorganization of Work: From Tayloristic to Holistic Organization,” *Journal of Labor Economics*, 2000, 18 (3), 353–76.
- Prescott, Edward C., Richard Rogerson, and Johanna Wallenius**, “Lifetime Aggregate Labor Supply with Endogenous Workweek Length,” *Review of Economic Dynamics*, 2009, 12 (1), 23–36.
- Shen, Judy Hanwan and Alex Tamkin**, “How AI Impacts Skill Formation,” 2026.
- Smith, Adam**, *An Inquiry into the Nature and Causes of the Wealth of Nations* 1776.
- Yang, Xiaokai and Jeff Borland**, “A Microeconomic Mechanism for Economic Growth,” *Journal of Political Economy*, 1991, 99 (3), 460–82.
- Zeira, Joseph**, “Workers, Machines, and Economic Growth,” *Quarterly Journal of Economics*, 1998, 113 (4), 1091–1117.

A Proofs

A.1 Limited proof of Proposition 2

Call an allocation $\{L_w^n\}$ *uniform-partitioning* if it partitions the set of workflows into N equal intervals, each performed exclusively by a single worker almost everywhere.

Given an allocation $\{L_w^n\}$, observe that the implied productivity schedules $\{A_w^n\}$ are continuous. For each n , let

$$H_w^n \equiv \int_0^w (w-v)^{-\beta} L_v^n dv$$

denote n 's "learning stock" at w , so $A_w^n = (H_w^n)^\gamma$. (We will sometimes write these schedules as functions of the allocation $L_{(\cdot)}^n$.)

Lemma 1: The optimal allocation of a worker over an interval. Assume here that there is only one worker, and drop the n superscripts. An allocation $L_{(\cdot)}$ across $(\underline{w}, \overline{w}]$ equalizes y_w across the range iff, for some k ,

$$\left(\int_{\underline{w}}^w (w-v)^{-\beta} L_v^n dv \right)^\gamma L_w = k \quad \forall w.$$

(We will quantify over $(\underline{w}, \overline{w}]$ unless otherwise specified.) Because the corresponding productivity schedule $A_{(\cdot)}$ is continuous, $L_{(\cdot)}$ must also be continuous.

Let $m_w \equiv L_w(w - \underline{w})^\beta$. After the change of variables $v \equiv \underline{w} + s(w - \underline{w})$ (from which $dv = (w - \underline{w})ds$),

$$\begin{aligned} H_w &= \int_{\underline{w}}^w (w-v)^{-\beta} m_v (v - \underline{w})^{-\beta} dv \\ &= \int_0^1 ((1-s)(w - \underline{w}))^{-\beta} (s(w - \underline{w}))^{-\beta} m_{\underline{w}+s(w-\underline{w})} (w - \underline{w}) ds \\ &= (w - \underline{w})^{1-2\beta} \int_0^1 ((1-s)s)^{-\beta} m_{\underline{w}+s(w-\underline{w})} ds. \end{aligned}$$

Letting J_w denote the integral of the last line above, so that

$$J_w = (w - \underline{w})^{2\beta-1} H_w,$$

constant y_w requires

$$((w - \underline{w})^{1-2\beta} J_w)^\gamma (w - \underline{w})^{-\beta} m_w = J_w^\gamma m_w = k. \quad (13)$$

Let

$$S \equiv \int_0^1 ((1-s)s)^{-\beta} ds, \quad \bar{m}_w \equiv \sup_{v \in (\underline{w}, w]} m_v, \quad \underline{m}_w \equiv \inf_{v \in (\underline{w}, w]} m_v.$$

Observe that $J_w \in [S\underline{m}_w, S\bar{m}_w]$. We will now show that there are $\underline{m} > 0, \bar{m} < \infty$ with $m_w \in [\underline{m}, \bar{m}] \forall w$.

Suppose not. Then, by continuity of $m(\cdot)$, there is either a sequence $\{w_k\} \rightarrow 0$ with $m_{w_k} \rightarrow 0$ or one with $m_{w_k} \rightarrow \infty$. In the latter case, if there is not also a sequence with $m_{w_k} \rightarrow 0$, then $\underline{m}_w$ and thus J_w is lower-bounded above 0, so (13) fails. Likewise, in the former case, if there is not also a sequence with $m_{w_k} \rightarrow \infty$, then $\bar{m}_w$ and thus J_w is upper-bounded, so (13) fails.

Suppose both sequences exist. Then for any $\varepsilon > 0$, there is a w_1 with $m_{w_1} = \varepsilon$ (and $J_{w_1} = (k/\varepsilon)^{-\gamma}$) and, by continuity, a $w_0 < w$ such that $m_{w_0} = k$ (and $J_{w_0} = 1$) and $m_w < k \forall w \in (w_0, w_1]$. Observe that, by definition of H ,

$$\begin{aligned} H_{w_1} &< H_{w_0} + \int_{w_0}^{w_1} (w_1 - v)^{-\beta} L_v dv \\ \implies (k/\varepsilon)^{1/\gamma} (w_1 - \underline{w})^{1-2\beta} &< (w_0 - \underline{w})^{1-2\beta} + \int_{w_0}^{w_1} (w_1 - v)^{-\beta} k(v - \underline{w})^{-\beta} dv \\ \implies (k/\varepsilon)^{1/\gamma} &< 1 + (w_1 - \underline{w})^{2\beta-1} \int_{\underline{w}}^{w_1} (w_1 - v)^{-\beta} k(v - \underline{w})^{-\beta} dv \\ &= 1 + (w_1 - \underline{w})^{2\beta-1} k \frac{(\Gamma(1-\beta))^2}{\Gamma(2-2\beta)} (w_1 - \underline{w})^{1-2\beta} \\ &= 1 + kS. \end{aligned}$$

But this fails for small ε . This establishes that $\exists \underline{m} > 0, \bar{m} < \infty : m_w \in [\underline{m}, \bar{m}] \forall w$.

For all w ,

$$k \leq m_w (\bar{m}_w S)^\gamma \leq m_w (\bar{m} S)^\gamma \quad \text{and} \quad k \geq m_w (\underline{m}_w S)^\gamma \geq m_w (\underline{m} S)^\gamma.$$

So, since for any $m_w > \underline{m}$ there is a corresponding w for which the first expression holds and for any $m_w < \bar{m}$ a corresponding w for which the second expression holds, we have

$$\begin{aligned}\underline{m} &\geq k(\bar{m}S)^{-\gamma}, \quad \bar{m} \leq k(\underline{m}S)^{-\gamma} \\ \implies \bar{m}\underline{m}^\gamma &\leq kS^{-\gamma} \leq \underline{m}\bar{m}^\gamma \\ \implies \underline{m} &\geq \bar{m}\end{aligned}$$

because $\gamma < 1$. So $m_w = m$ is constant.

To find m , subject $L_w = m(w - \underline{w})^{-\beta}$ to the unit labor constraint:

$$\int_{\underline{w}}^{\bar{w}} L_w dw = 1 \implies m = (1 - \beta)(\bar{w} - \underline{w})^{\beta-1}. \quad (14)$$

Substituting into (12) yields

$$\begin{aligned}A_w &= \left(m \int_{\underline{w}}^w ((w - v)(v - \underline{w}))^{-\beta} dv \right)^\gamma \\ &= \left(m \frac{(\Gamma(1 - \beta))^2}{\Gamma(2 - 2\beta)} (w - \underline{w})^{1-2\beta} \right)^{\frac{\beta}{1-2\beta}} \propto (w - \underline{w})^\beta, \end{aligned} \quad (15)$$

confirming that $A_w L_w$ is constant.

Granting (i) and (iii), this implies that the allocation above is the unique optimal allocation of a worker over an interval.

The optimal allocation, disjoint case. Suppose $\{L_{(\cdot)}^n\}$ is disjoint but not uniform-partitioning. Let σ^n denote the support of L^n . Let L_w^* be the optimal allocation of a unit of labor across the interval $(0, 1/N]$. Let $\{L_{(\cdot)}^{*n}\}$ denote the uniform-partitioning allocation with $L_w^n = L_{w-(n-1)/N}^*$. Let $\mu(\cdot)$ denote the Lebesgue measure.

Suppose that for all n ,

$$L_w^n = L_{\mu([0, w] \cap \sigma^n)}^* \quad \forall w \in \sigma^n. \quad (16)$$

Note the implication that $\mu(\sigma^n) = 1/N$ for all n . By the assumption that $\{L_w^n\}$ is not uniform-partitioning, we have that for some n and $\underline{w}$, $S \equiv [\underline{w}, 1] \cap \sigma^n$ is positive

measure and A_w^n is smaller throughout S —and thus $A_w^n L_w^n$ is smaller a.e. in S —than if σ^n were an interval but (16) were maintained. Thus $Y(\{L_{(\cdot)}^{*n}\}) > Y(\{L_{(\cdot)}^n\})$.

Suppose that, for some m , (16) fails with respect to all optimal $\{L_{(\cdot)}^*\}$. Then let $\{\bar{L}_{(\cdot)}^n\}$ denote a *compression* of $\{L_{(\cdot)}^n\}$: an allocation with supports $\{\bar{\sigma}^n\}$ that partition the interval, maintaining for all n

$$\bar{L}_{\mu([0,w] \cap \bar{\sigma}^n)}^n = L_{\mu([0,w] \cap \sigma^n)}^n \quad \forall w \in \bar{\sigma}^n. \quad (17)$$

Note the implication that $\mu(\bar{\sigma}^n) = \mu(\sigma^n)$ for all n , and that because every workflow in $\bar{\sigma}^n$ can be mapped to a workflow in σ^n with equal labor but weakly less effective labor, $Y(\{\bar{L}_{(\cdot)}^n\}) \geq Y(\{L_{(\cdot)}^n\})$. Now observe that either $\mu(\bar{\sigma}^m) > 1/N$ for some m or $\mu(\bar{\sigma}^n) = 1/N$ for all n . In the first case, since the highest infimum achievable over a wider interval is less than that over a narrower interval, it follows immediately that $Y(\{L_{(\cdot)}^{*n}\}) > Y(\{\bar{L}_{(\cdot)}^n\})$. In the second case, $\{\bar{L}_{(\cdot)}^n\}$ is uniform-partitioning; but by the assumption that (16) fails for some n , $Y(\{L_{(\cdot)}^{*n}\}) > Y(\{\bar{L}_{(\cdot)}^n\})$.

Granting (ii), this implies that the allocation in the statement of Proposition 2 is uniquely efficient, up to relabeling of the workers.